

Comparing Vision Transformers and CNNs for Sex Classification from Handwriting

Abstract. Handwriting is a biometric marker, encoding information regarding an individual’s demographic traits, psychological state, and neurological health. Advances in machine learning enables the automatic analysis of handwriting, with potential applications in fields like forensic and healthcare. Pre-trained deep learning vision models are often used instead of training new models from scratch, because of the limited availability of data for handwriting classification tasks. Vision Transformers (ViTs) are increasingly adopted as pre-trained baselines in many applications, however, their application to handwriting classification remains under-explored. To address this, we present a comparative analysis of ViTs and CNNs architectures as feature extractors for the task of sex classification, using the publicly available ICDAR2013 "Gender Prediction from Handwriting" dataset. We explore a wide array of pre-trained models, differing in architecture, pre-training data and pre-training objectives. Our results show that models based on ViTs features consistently outperform those based on CNNs features on this task. Exploiting features extracted from clip-vit-large [37] we are able to achieve a page-level classification accuracy of 80.1% comparable with the best results achieved in the literature with more task specific approaches.

Keywords: Gender / Sex Detection · Forensics · ViTs · Handwriting

1 Introduction

It is known that handwriting characteristics can be influenced by medical conditions like Parkinson’s disease (PD) and Alzheimer’s disease (AD) [10], by psychological conditions like depression [26] and by demographical characteristics such as sex and age [2]. Machine learning (ML) systems have been developed for assessing health status or demographical characteristics of the writer from handwriting samples, with applications in healthcare or other domains. These classification tasks require analysis of handwriting at both the local and global level and are thus in between handwriting recognition and writer identification tasks [2].

Sex detection from handwriting is one of the tasks for which there are more publicly available datasets and for which it is easier to collect labelled data. It is interesting because it may be useful in areas such as paleography, forensic analysis, biometric identification systems and forgery detection [2]. For example in paleography, identifying the sex of a writer in a manuscript could be useful for reconstructing the social and human context, e.g. we know if the manuscript comes from a monastery or a nunnery, while in forensics it may be used to narrow

the suspect pool in presence of handwritten notes or as an element for victim identification [33].

Numerous studies show that humans seem able to distinguish male and female handwriting [5,6]. ML approaches for sex detection reach accuracies around 80% with different techniques [34,46,47] and some engineered features seems to correlate strongly with sex [1,14,20] providing evidence for the influence of sex on handwriting. Some studies inquire more in depth about the differences between female and male handwriting and the mechanisms that may explain these differences. Qualitative analyses describe female as more 'clean', 'elegant' and 'regular' [6,14,33]. Quantitative analyses show differences in the speed and fluency of handwriting [9,32] and features like pressure, jerk and so on [13,39]. Sex detection from handwriting has been tackled via different ML approaches that attempt to cope with the scarce data problem, including feature extraction approaches and deep learning approaches based on pre-trained models [2]. The state of the art in deep learning architectures for computer vision has been changing, for instance Vision Transformers (ViTs) have been shown to outperform Convolutional Neural Networks (CNNs) across various computer vision benchmarks and thus they are increasingly adopted as pre-trained baselines in many applications. These changes have not transferred to the sex detection and handwriting classification tasks in which CNNs are still the most widely used architecture and ViTs are not yet widely adopted. In this paper we focus on the task of sex detection from handwriting. We focus on two research questions: how do ViTs compare with CNNs as feature extractors for the sex detection task from handwriting? Which properties of the pre-trained models and which pre-processing strategies have effect on the quality of the extracted representations for the task? In order to answer them, we present a comparative analysis of pre-trained ViTs and CNNs architectures for the sex detection task using the publicly available ICDAR2013 "Gender Prediction from Handwriting" dataset [15]. We explore a wide array of pre-trained models, differing in architecture, pre-training data and pre-training objectives. Moreover, we propose an approach to exploit pre-trained models without model tuning, for the detection of sex from handwritten text. We evaluate it and give insights on the best strategies for pre-processing the input data, for extracting the representations from the pre-trained models and for exploiting the representations for the final classification.

2 Related Works

Most previous attempts at sex detection on the ICDAR2013 dataset [15] exploited the feature extraction methodology. In this type of approach, the researcher decides which features to extract from the images and a shallow ML model uses them to classify the images. Some relevant results are reported in papers like [1,14,20], reviews [2] and the original competition paper [15]. The reported accuracies on the dataset for these methods range between 60% and 79%, showing that these approaches can be very effective. Another approach consists in using deep learning models without defining pre-specified features

giving accuracies up to 85% [46]. The problem of data scarcity can be addressed by crafting a task-specific CNN architecture and training it from scratch [47]. It can be addressed by fine-tuning pre-trained CNN architectures [46], like the resnet50 [16] and the vgg19 net [41], or by using pre-trained models only as feature extractors [34,36], as we do here. In [36] many CNN architectures are compared as feature extractors. This work is the one that aligns most closely to ours. The only work that exploits ViTs on the ICDAR2013 dataset is [21], but for writer identification.

Other datasets often cited in the literature on sex detection from handwriting are QUWI [29] (of which the ICDAR 2013 dataset is part), IAM [31], KHATT [30] and HHD [35]. The deep learning approaches tested on them mostly involve fine-tuning models pre-trained on ImageNet [2] as for the ICDAR 2013 dataset.

The methods reported above also differ in the strategy with which the pages are preprocessed. Using full pages for detection is the standard strategy. Splitting in patches, classifying each patch and aggregating results [34,36] is the one we follow. Splitting in patches, extracting representations, aggregating and classifying on the aggregated representation is another possibility [21,47].

To the best of our knowledge, there is no direct application of vision transformer models to the task of sex detection from handwriting, but research on similar tasks suggests that these models may perform well. Moreover, the field lacks a comparison of pre-trained models that includes different architectures, pre-training tasks, and pre-training data. Most research papers only consider CNN models pre-trained on ImageNet or trained from scratch on the target dataset.

3 Methods

The general approach we propose for predicting the sex from a page of handwritten text is exemplified in figure 1. The image is splitted in patches, each patch is converted in a 1-dim vector (representation or feature vector) by a pre-trained model. Each representation is then passed through a classifier model that returns a probability for the patch to be from a male or a female and finally the probabilities for the patches from a same image are combined to give a page-level prediction. Thus the only element of this pipeline that has to be learned from the data is the classifier, the other elements are fixed and not trained on the target dataset (ICDAR2013 in our case). A prediction model is thus defined by the way in which pages are split in patches and in which the relevant patches are selected (page-splitting methodology), by the pre-trained model used to extract the representations from the patches and the strategy used for extracting the representations from the pre-trained model (representation extraction methodology), by the type of classifier used and by the strategy used for aggregating the classifier predictions (aggregation methodology).

Felsic lavas (dacites or rhyolites) tend to be highly viscous (or very fluid) and are erupted as domes or short, stubby flows. Viscous lavas tend to form stratovolcanoes or lava dome-tension peaks. In California is an example of a volcano formed from felsic lava and is actually.

3.1 Dataset Description

For each writer, we have four pages of handwritten text. The first and second page are written in English, while the third and fourth in Arabic. The text for the first and third page is the same for all writers while the text of the second and fourth page changes from writer to writer as shown in the left panel of figure 2.

3.2 Selected Pre-trained Models

We test different types of pre-trained models as feature extractors. We test models that differ in the size of the dataset used for pre-training them, in the modality of pre-training, in the type of data used for pre-training and in the type of architecture, so as to explore which of these factors most influences the quality of the representations extracted for the sex detection task.

All tested models are described in table 1. We consider convolutional neural networks (CNNs) and vision transformer (ViT) architectures. Within these two types of architectures we have considered different families of models. Each family includes many models that differ in depth and number of parameters, but share the same architectural choices. Most model families in the transformer class are based on the same vision transformer base architecture [12] and differ only in the pre-training, while CNN families differ in the architecture.

Some pre-training strategies exploited to train the models are supervised learning approaches: classification (Cl in table 1), classification+distillation (Cl+Di), where the distillation procedure [17] is used to train a model for classification using another teacher model, text-recognition (TR) and text detection (TD). Other pre-training strategies are unsupervised: contrastive pre-training (Co) [8] and masking (Ma), where the models try to reconstruct a missing portion of the input image during pre-training.

The models are trained on one of five data sources. The "trocr" dataset is a collection of typed text synthetically generated and crawled from the web [24]. In some models, a fine tuning on the IAM dataset follows a pre-training on the trocr set, "trocr+IAM" (T+I in table 1). The "doctr" dataset is a collection of many datasets for scene text detection, scene text recognition, document understanding and other related tasks <https://mindee.github.io/doctr/latest/modules/datasets.html>. Most models are trained on the well known "ImageNet1k" (IN1K) and "Imagenet21k" (IN21K) datasets [11]. Most images in these datasets do not contain typed text, handwriting or scene text. The clip models are the only ones trained on a different dataset which we call "Clip". It is a large collection of images and corresponding captions crawled from the web and grouped from different publicly available datasets <https://huggingface.co/openai/clip-vit-large-patch14>.

To represent CNN architectures pre-trained on non text-like data, we have considered all the CNNs families present on the torch model repository <https://docs.pytorch.org/vision/main/models.html>. To represent ViT architectures pre-trained on non text-like data, we have selected the ViT model families from huggingface with the highest performance on the ImageNet benchmark. To make a comparison with CNNs and ViT models pre-trained on text-like data, we looked for models pre-trained or fine-tuned for these specific tasks on huggingface and on doctr and included the most relevant ones. For each of these families, we considered models with different size and depth and thus we selected a small, medium and large version (if available).

Some models in table 1 have the label "-inter" or "_layerN" (e.g. convnext_large and convnext_large-inter). The version with the label is one in

which we use the representation extracted at an intermediate layer of the encoder and not at the layer right before the classification head or decoder (see 3.4). Intermediate representations should contain low level information on the input (edges, textures, corners, ...) and may be more useful for tasks related to handwriting classification, where local features are very important.

Table 1: Classes of pre-trained models considered in the study. The first three columns report the architecture type (Arch), the pre-training strategy (Pre-train), and the dataset used for pre-training (Data). The last column lists the individual pre-trained models included in each class. N-par denotes the range of the number of model parameters.

Arch	Pre-train	Data	N-par	Models
CNN	CI	IN1K	[2.3e+05, 6.4e+08]	alexnet [22], convnext_small [28], convnext_base, convnext_large, convnext_large-inter, densenet121 [19], densenet161, densenet201, efficientnet_v2_s [44], efficientnet_v2_l, googlenet [42], inception_net [43], mobilenet_v3_small, mobilenet_v3_large [18], regnet_x_3_2 [38], regnet_x_32, regnet_y_3_2, regnet_y_32, regnet_y_128, resnet18, resnet34, resnet34_layer1 [16], resnet34_layer2, resnet34_layer3, resnet50, resnet101, vgg11 [41], vgg16
CNN	TD	Doctr	[3.0e+06, 2.4e+07]	db_mobilenet [25], dresnet50 [25], dresnet50-inter, linknet_resnet18 [7], linknet_resnet50
CNN	TR	Doctr	[1.3e+05, 4.6e+07]	crnn_mobilenet [40], crnn_mobilenet-inter, crnn_vgg16 [40], crnn_vgg16-inter, sar_resnet31 [23]
ViT	CI	IN1K	[4.9e+07, 8.7e+07]	swin_b [27], swin_s
ViT	CI	IN21K	[8.6e+07, 6.3e+08]	vit-base-patch16-224 [12], vit-base-patch16-224-in21k, vit-base-patch32-224-in21k, vit-huge-patch14-224-in21k, vit-large-patch16-224-in21k
ViT	CI+Di	IN1K	[5.5e+06, 8.7e+07]	DeiT-Base [45], DeiT-Small, DeiT-Small-Dist, DeiT-Tiny, DeiT-Tiny-inter
ViT	Co	Clip	[1.5e+08, 4.3e+08]	clip-vit-base-patch16 [37], clip-vit-base-patch32, clip-vit-large-patch14, clip-vit-large-patch14-inter
ViT	Ma	IN21K	[8.7e+07, 3.0e+08]	BEiT-Base [4], BEiT-Large, BEiT-Large-inter
ViT	TR	Doctr	[2.1e+07, 8.5e+07]	vitstr_base [3], vitstr_small
ViT	TR	Trocr	[2.2e+07, 3.0e+08]	trocr-base-stage1 [24], trocr-large-stage1, trocr-large-stage1-inter, trocr-small-stage1
ViT	TR	T+I	[2.2e+07, 3.0e+08]	trocr-base-handwritten [24], trocr-large-handwritten, trocr-large-handwritten-inter, trocr-small-handwritten

3.3 Page-Splitting Strategies

We compare two different strategies for splitting the page in patches and selecting the relevant patches to be processed by the pre-trained model. The first strategy is the trivial one in which the full page is used as input for the pre-trained model, thus we have one patch per page. We call this strategy "body level selection". The second strategy is to define a squared moving window with fixed size and stride and select all the patches that can be reached by moving the window

across the page. We can have a different number of patches per page, a different patch size, and a different amount of overlap between patches by varying the parameters of the window. If we apply this method, some of the patches may be empty or contain very little text, thus we apply an exclusion criterion to eliminate these patches. Namely we count the number of connected components in the binarized patch and exclude the patch if we have no connected components (we are excluding most non text patches). We then count the number of black pixels in the binarized patches, we sort them according to the number of black pixels and select the patches with the higher percentage of black pixels (we are roughly selecting the images with most text). We call this approach "standard squares" when the patch size is fixed to $1/5$ of the horizontal resolution, the stride is fixed to the patch size (thus no overlap) and the number of patches selected per page is 5. The patch size is chosen based on 3 while the number of patches is restricted to five to extract faster the representation in the comparison experiments and to guarantee a sufficient number of patches for aggregation.

3.4 Extracting Patch Representations with Pre-trained Models

Each patch is processed by a pre-trained model to extract a representation. Each pre-trained model requires the input image to be processed in a specific way (e.g. resized to a specific resolution, normalized with specific mean and standard deviation and so on) thus the first step is to apply the required pre-processing to the patches. The second step is to perform inference on the pre-trained model and exploit the output of the model to produce a 1-dim representation for each patch. The way in which the output of the pre-trained model is used to produce the representation depends on the architecture of the model and on the way in which it was pre-trained.

The pre-trained models have one of the following structures (as in figure 1): "vision-encoder"+"decoder", "vision-encoder"+"classification-head" or "vision-encoder"+"other-encoders" structure. We always consider the vision-encoder and discard the classification head or the decoder part of the model and the other encoders.

In VIT-based models the output of each layer of the model is a sequence of $k+1$, n -dimensional, tokens. The last k tokens encode k patches of the original image while the first token (cls token) aggregates the information from the other tokens for classification purposes [12]. We take the cls token from a specific layer as the extracted model representation. In CNN encoders the output of the convolutional layers has shape (n, h, w) (n channels of $h \times w$ dimension). Since we want a one dimensional output from the model we apply adaptive 2D average pooling to reduce the dimension to $(n, 1, 1)$ and then flatten the tensor to extract a representation from a convolutional layer. The output of the final layers of a CNN encoder are instead fully connected layers with 1-dim vector output, thus, if we use these layers, we just take the output of the layer as the representation without performing any other operation on it.

In our experiments, we extract representations from different layers of the pre-trained models, including intermediate layers and the layer right before the

classification head or decoder. If an intermediate layer is used, we specify it in the name of the model with the label "-inter" or "_layerN" (see table 1).

As a final step we normalize the representations extracted from the models.

3.5 Training the Classifier

After obtaining a representation for all patches by using the pre-trained model, we train a simple classifier, such as a logistic regression, a decision tree, or an MLP on the representations dataset to classify each representation vector (each patch) as from a man or from a woman. This approach maximizes the number of training samples for the classifier, since we have as many samples as patches instead of one sample per page.

3.6 Aggregating Patch-level Predictions

We propose multiple ways of aggregating the patch-level predictions of the classifier in a page level prediction. Let p_{ij} $j = 1, \dots, m$ ($0 \leq p_{ij} \leq 1$) be the output of the classifier for the j th patch of the i th page. Let g_{ij} be the classification result for the patch ($g_{ij} = 1$ if $p_{ij} \geq 0.5$ and $g_{ij} = 0$ otherwise). Finally let $c_{ij} = |p_{ij} - 0.5|/0.5$ be the confidence of the prediction on a patch. Then, in our experiments, we compute a page level prediction G_i with one of the following approaches:

- Majority vote: $G_i = \text{round}\left(\left(\sum_j g_{ij}\right)/m\right)$; the prediction that appears most frequently in the patch level predictions is selected as page level prediction.
- Most probable: $G_i = g_{ik}$ with $k = \text{argmax}_j |p_{ij} - 0.5|/0.5$; the most confident prediction at patch level is used as page level prediction.
- Weighted vote: $G_i = \text{round}\left(\sum_j c_{ij}g_{ij}\right)$; Averaging the patch level predictions according to the confidence on the prediction for each patch.

In the analysis, writer-level predictions are always computed by majority voting across pages: $G_k = \text{round}\left(\left(\sum_{ij} g_{kij}\right)/(4 \cdot m)\right)$ where k is the writer index and i iterates on the four pages for the k th writer.

3.7 Procedure to Determine and Evaluate the Best Pipeline

Since our sex detection model is really a pipeline of different elements (page splitting method, pre-trained model and representation extraction method, classification model and aggregation method) we have to make good choices for each element of the pipeline.

We will first use the public train set for understanding how to perform the page-splitting, which pre-trained models to use and which representation extraction approaches to use. In this phase, we fix the aggregation method to weighted vote (see 3.6) and the classification model to the logistic regression. The logistic regression is a good baseline for comparing different feature extraction methods

(often indeed you perform a linear evaluation to compare model representations). It is also fast to train and enables to perform a k-fold cross validation to have more robust results. We perform k-fold cross validation on the public train set for each configuration of page-splitting method and pre-trained model, and we compare the OOF accuracy for each configuration to gain insights on the strategies that work better.

With these insights we fix a page-splitting strategy and a set of best pre-trained models to use. For each pre-trained model we train it on the public train using the selected splitting strategy and we evaluate it on the public test set (again considering linear evaluation and weighted vote ensembling) using hold-out validation. We select only the best three pre-trained models and another one for reference with the literature (selected models are reported in section 4.3).

Once these four pre-trained models are selected and the page-splitting strategy is fixed we use the public train set and the public test set for choosing the best classifier model, its hyperparameters and the best aggregation method for each pre-trained model, via hold-out validation. This gives us one pipeline (or configuration) for each pre-trained model.

We will then test the chosen configurations on the private test set to evaluate the performance of the sex detection pipeline.

When performing cross-validation and hold-out validation, we always split the data at the writer level, ensuring that patches from the same writer are used exclusively for training, validation, or testing, but never simultaneously across multiple splits. We use accuracy at page-level as the main metric to evaluate the models performance to remain consistent with the literature.

4 Results and discussion

4.1 Pre-trained Model Comparison

To compare the pre-trained models, we follow the approach in 3.7 using 10-fold cross-validation. We report the results in table 2. The Acc is the accuracy of the patch-level predictions, while w-Acc is the accuracy at the page-level. Note that w-Acc and Acc are computed after using the "standard squares" approach for patch selection.

The models in the table are grouped by architecture and pre-training task, and in each class they are sorted in descending order according to w-Acc. The classes are also ordered in descending order of the w-Acc value reached by the best model in the class. For each class, we only report the best model in the class and their corresponding "-inter" or standard version (if available), for comparison. We also add some other models for comparison purposes (labelled with an asterisk in the table).

We observe that the accuracy for the models in the transformer class is higher than the accuracy of CNN models, with only the ConvNext giving close results. Thus it seems that ViT models give better representations than CNNs.

We see that the best performing models in each model family are the largest models in the family (e.g. see BeiT, clip-vit, convnext and so on) apart for the

DeiT family. Since models in the same family are trained in the same way, with the same architecture and on the same dataset, it is the model depth and number of parameters that makes a difference. Thus, model size seems to have a positive impact on the performance for this specific task. We can also see that the "-inter" version seems to give a better accuracy for most models. Hence, using representations from layers closer to the input may lead to better performance, as previously hypothesized.

If we compare CNN models pre-trained for classification (resnet18, resnet50 and vgg16) with the corresponding detection and recognition models trained on doctr data, we see that the doctr models have better accuracy. As expected CNN models fine-tuned on data more similar to the downstream task's data type (sex detection in this case) provide better representations.

The trocr-base models are initialized from the weights of the BEiT-Base model and are fine tuned on trocr data for text recognition [24]. The same holds for trocr-large and BEiT-Large. But we see that fine-tuned models are, unexpectedly, worse for sex detection. Also, comparing all architectures based on vit-base-patch16 (clip-vit-base-patch16, DeiT-Base, Beit-Base, vit-base-patch16 and trocr-base-stage1) it seems that masking is the pre-training approach that gives the best representations for sex detection.

4.2 Patch-Splitting Method Comparison

Table 2 also reports the accuracies s-Acc and b-Acc. s-Acc is computed with the standard squares selection approach but keeping a single patch per page for training the classifier instead of five. b-Acc is obtained by training with the standard body selection approach. Comparing Acc and s-Acc we can see that using more patches per page increases the model accuracy and comparing s-Acc and b-Acc we can see that with an equal number of training samples training on smaller patches or considering the full page gives similar results. Maybe, even if the full page has more information, a smaller patch forces the model to focus on more local patterns, generating a tradeoff.

The cross-validation allows us to investigate how accuracy depends on patch size and number of selected patches. As an example, Fig. 3 shows the results when the clip-vit-large-patch14 pre-trained model is used as feature extractor (the trends remain the same for other pre-trained models). We see that the OOF accuracy increases with the number of patches while the IF accuracy decreases. Combining this observation with the fact that data augmentation was not found to be useful for improving classifier performance we can conclude that having multiple patches per page acts as a form of data augmentation and helps reducing overfitting. We also see that the individual level accuracy plateaus after a certain point while the ensembled accuracy does not. Of course, the benefit of ensembling patch-level predictions increases with the number of available patches (at least with the weighted ensembling approach).

Concerning the dependence on patch size, we observe that the accuracy and generalization gap are optimal at around 1/5 page size decreasing from then on. However, the individual patch accuracy is still above chance (actually around

Table 2: Pre-trained models with highest page-level accuracy (w-Acc) under linear evaluation, divided by class. The symbol * highlights models that are added for comparison purposes but are not the best performing models in their class. N is the number of models from table 1 tested for the class. N-par gives the number of parameters of the vision encoder of the pre-trained model. s-Acc gives the accuracy when training on a single patch per page and b-Acc when training on full pages (body).

Model	Arch	Pre-train	N	N-par	w-Acc	Acc	s-Acc	b-Acc
clip-vit-large-patch14-inter	ViT	Co	4	3.0e+08	0.774	0.740	0.735	0.723
clip-vit-large-patch14	ViT	Co	4	3.0e+08	0.741	0.686	0.650	0.649
clip-vit-base-patch16*	ViT	Co	4	8.6e+07	0.730	0.678	0.639	0.653
BEiT-Large	ViT	Ma	3	3.0e+08	0.768	0.719	0.657	0.658
BEiT-Large-inter	ViT	Ma	3	3.0e+08	0.766	0.730	0.717	0.732
BEiT-Base*	ViT	Ma	3	8.7e+07	0.765	0.711	0.677	0.665
trocr-large-handwritten-inter	ViT	TR	10	3.0e+08	0.753	0.694	0.669	0.672
trocr-large-handwritten	ViT	TR	10	3.0e+08	0.712	0.664	0.618	0.643
trocr-base-stage1	ViT	TR	10	8.7e+07	0.710	0.679	0.662	0.618
convnext_large	CNN	Cl	28	2.0e+08	0.735	0.699	0.677	0.689
efficientnet_v2_s	CNN	Cl	28	2.0e+07	0.728	0.690	0.691	0.656
convnext_large-inter	CNN	Cl	28	4.8e+06	0.728	0.700	0.684	0.678
resnet50*	CNN	Cl	28	2.4e+07	0.689	0.643	0.617	0.638
vgg16*	CNN	Cl	28	1.3e+08	0.642	0.599	0.575	0.625
resnet18*	CNN	Cl	28	1.1e+07	0.640	0.597	0.586	0.583
DeiT-Small	ViT	Cl+Di	5	2.2e+07	0.730	0.674	0.670	0.669
DeiT-Base*	ViT	Cl+Di	5	8.7e+07	0.698	0.651	0.666	0.660
DeiT-Tiny-inter	ViT	Cl+Di	5	5.5e+06	0.690	0.663	0.628	0.648
DeiT-Tiny	ViT	Cl+Di	5	5.7e+06	0.684	0.647	0.649	0.656
swin_s	ViT	Cl	7	3.3e+07	0.730	0.687	0.688	0.662
vit-base-patch16-224-in21k*	ViT	Cl	7	8.6e+07	0.713	0.667	0.628	0.672
linknet_resnet18	CNN	TD	5	1.1e+07	0.717	0.684	0.688	0.677
dresnet50	CNN	TD	5	2.4e+07	0.715	0.663	0.650	0.685
dresnet50-inter	CNN	TD	5	2.4e+07	0.709	0.684	0.652	0.699
crnn_vgg16-inter	CNN	TR	5	1.7e+06	0.704	0.681	0.652	0.659
crnn_vgg16	CNN	TR	5	1.5e+07	0.680	0.642	0.635	0.619
sar_resnet31	CNN	TR	5	4.6e+07	0.648	0.626	0.620	0.602

60%) even for very small patches. These patches cannot have information on character spacing or line directionality but only information at character or sub-character level, suggesting that sex-related information is present also at character and sub-character level.

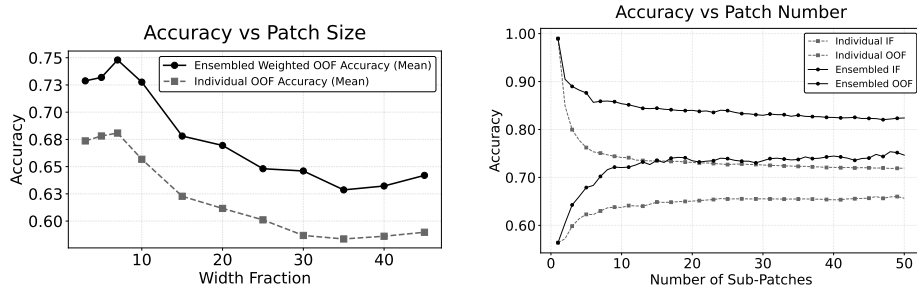


Fig. 3: In-fold and out-of-fold accuracies for different patch-splitting parameters are reported for the clip-ViT-large-patch14 pre-trained model. Both single patch and aggregated page-level accuracies are shown. On the left we change the size of patches fixed the number of patches (one over the width fraction is the ratio between the patch size and the page width). On the right we keep the patch size fixed and we change the number of patches.

4.3 Building the Pipeline and Final Evaluation

Given the above discussion we fix the patch-splitting strategy to a method that maximizes the number of patches without reducing too much their dimension. From each page, we extract 40 patches (224×224 px) using a stride of 122 (see 3.3), overlap is needed for reaching the target number of patches.

We select the best pre-trained model in each of the classes reported in table 2 as our initial set of models and we evaluate the performance of these models on the validation set using the new patch-splitting approach. This gives us clip-vit-large-patch14-inter, clip-vit-large-patch14 and BEiT-Large as the three best pre-trained models and we also keep the resnet50 model for comparison with previous works as explained in 3.7.

For each we select the classifier, its hyperparameters and the aggregation method as explained in 3.7. After the hyperparameter tuning we have four pipelines with fixed patch-splitting, pre-trained model, classifier and aggregation method. We note that the chosen classifier is always an MLP with one hidden layer. For each pipeline the results on the test set are reported in the table 3 and compared with other state of the art methods from the literature.

We can also improve the performance of the selected model by performing a calibration of the classifier’s outputs on the validation set. We report the results with and without calibration only for the ‘clip-vit-large-patch14-inter’ model in the table 3. Calibration is performed using sklearn isotonic calibration.

Experiments were conducted on a Dell Inspiron 16 with an NVIDIA GeForce RTX 4060. While top-model performance is largely robust to hyperparameter choices—provided strong regularization like aggressive early stopping is used—our optimal configurations are available in the project repository https://github.com/gitMorelli/gender_detection.git.

4.4 Comparison with Results from the Literature

The results of the evaluation of our best models on the test set are reported in table 3 and compared with the results on other methods from the literature. We report the page level accuracy when the model is trained on both arabic and english and tested on both (Ar+En/Ar+En), the page level accuracy when the model, with the same hyperparameter choices, is trained on english and tested on english (En/En), when is trained on Arabic and tested on Arabic (Ar/Ar) and when trained on English and tested on Arabic (En/Ar). Our best model is the one with the best (En+Ar/En+Ar) accuracy at the page level, second only to the results achieved in [46]. However, we highlight a problem in the way results are reported in the literature. Many works do not provide details about their evaluation procedure. For instance most works do not specify if the train-test split is performed at the page-level or at the writer-level, but splitting at the page-level may artificially increase the reported performance because of leakage of information on writer style. In table 3, we use the "Split" column to specify if the train-test split is performed at the page-level or at the writer-level or if this information is not provided. For completeness we add that, for our best model "clip-vit-large-patch14-inter (cal)", the patch-level accuracy is 74.4% and the writer-level accuracy is 83.6%. We cannot do a comparison with the literature since most paper only report the page-level accuracy (see 3.6).

Table 3: Evaluation results on the test set for our best models and comparison with the literature. Page column is equivalent to En+Ar/En+Ar.

Method	Page	En/En, Ar/Ar, En/Ar	Split
Ours, clip-vit-large-patch14-inter	0.787	0.832, 0.754, 0.619	Writer
Ours, clip-vit-large-patch14-inter (cal)	0.801	0.840, 0.791, 0.758	Writer
Ours, clip-vit-large-patch14 (cal)	0.762	0.799, 0.754, 0.611	Writer
Ours, BEiT-Large-inter (cal)	0.775	0.816, 0.746, 0.717	Writer
Ours, ResNet50 (cal)	0.713	0.766, 0.668, 0.680	Writer
Handcrafted features [14]	0.714	0.780, 0.762, 0.694	Writer
Handcrafted features [1] (ANN)	0.793	0.690, 0.719, -	Writer
Handcrafted features [20]	0.710	-	Unknown
Handcrafted features, winning icdar [15][34]	0.76	0.79, 0.74, -	Writer
Feature extraction with CNNs [36] (Resnet50)	0.670	0.670, 0.500, -	Writer
Feature extraction with CNNs [36] (Best)	0.71	0.75, 0.74, -	Writer
Specific architecture, atp-densenet [47]	0.718	-	Unknown
Specific architecture, B-ResNet [34]	0.780	0.883, 0.852, -	Writer
Fine-tuning, Resnet50 [47]	0.687	-	Unknown
Fine-tuning, Resnet50 [46]	0.848	-	Unknown

5 Conclusions

We found evidence that ViT-based models can provide better representations for the sex detection task with respect to CNN-based models. Additionally, our results suggest that the size of the model, the pre-training task and the pre-training data have an influence on the usefulness of the representations.

We have proposed a pipeline that exploits patch splitting and prediction aggregation to make good use of the representations extracted from pre-trained models. We have shown that factors such as patch size and number of selected patches have an important effect on sex detection accuracy.

We have trained a best model following this method, achieving an accuracy of 0.801 at the page level on the private test set, outperforming most previous works in the literature and showing the effectiveness of the proposed approach.

We think it may be useful for researchers to consider some of our most relevant findings when tackling gender detection tasks or similar tasks. We suggest that using pre-trained models as feature extractors can suffice without fine-tuning. We recommend evaluating intermediate layer representations, and utilizing image patches rather, or with, the full images, to force focus on local details. Crucially, researchers must precisely document their evaluation procedures—particularly cross-writer data splitting—to ensure cross-study comparability. Future work should address: improving aggregation methodologies, evaluating the efficiency and impact of large pre-trained models, and applying interpretable ML techniques to study representation and learned patterns.

References

1. Akbari, Y., Nouri, K., Sadri, J., Djeddi, C., Siddiqi, I.: Wavelet-based gender detection on off-line handwritten documents using probabilistic finite state automata. *Image and Vision Computing* **59**, 17–30 (2017). <https://doi.org/https://doi.org/10.1016/j.imavis.2016.11.017>
2. Alaei, F., Alaei, A.: Review of age and gender detection methods based on handwriting analysis. *Neural Computing and Applications* **35**(33), 23909–23925 (2023). <https://doi.org/10.1007/s00521-023-08996-x>
3. Atienza, R.: Vision transformer for fast and efficient scene text recognition. In: *Proc. 16th Int. Conf. on Document Analysis and Recognition (ICDAR)*. pp. 319–334. Springer (2021). https://doi.org/10.1007/978-3-030-86549-8_21
4. Bao, H., Dong, L., Piao, S., Wei, F.: BEit: BERT pre-training of image transformers. In: *International Conference on Learning Representations* (2022), <https://openreview.net/forum?id=p-BhZSz59o4>
5. Beech, J.R., Mackintosh, I.C.: Do differences in sex hormones affect handwriting style? evidence from digit ratio and sex role identity as determinants of the sex of handwriting. *Personality and Individual Differences* **39**(2), 459–468 (2005). <https://doi.org/https://doi.org/10.1016/j.paid.2005.01.024>
6. Burr, V.: Judging gender from samples of adult handwriting: Accuracy and use of cues. *The Journal of Social Psychology* **142**(6), 691–700 (2002). <https://doi.org/10.1080/00224540209603929>

7. Chaurasia, A., Culurciello, E.: Linknet: Exploiting encoder representations for efficient semantic segmentation. In: 2017 IEEE Visual Communications and Image Processing (VCIP). pp. 1–4 (2017). <https://doi.org/10.1109/VCIP.2017.8305148>
8. Chen, T., Kornblith, S., Norouzi, M., Hinton, G.: A simple framework for contrastive learning of visual representations. *International Conference on Machine Learning* **1**, 1597–1607 (2020), <http://proceedings.mlr.press/v119/chen20j/chen20j.pdf>
9. Cohen, M.R.: Individual and sex differences in speed of handwriting among high school students. *Perceptual and Motor Skills* **84**(3_part_2), 1428–1430 (1997). <https://doi.org/10.2466/pms.1997.84.3c.1428>
10. De Stefano, C., Fontanella, F., Impedovo, D., Pirlo, G., Di Freca, A.S.: Handwriting analysis to support neurodegenerative diseases diagnosis: A review. *Pattern Recognition Letters* **121**, 37–45 (2018). <https://doi.org/10.1016/j.patrec.2018.05.013>
11. Deng, J., Dong, W., Socher, R., Li, L.J., Li, K., Fei-Fei, L.: Imagenet: A large-scale hierarchical image database. In: 2009 IEEE Conference on Computer Vision and Pattern Recognition. pp. 248–255 (2009). <https://doi.org/10.1109/CVPR.2009.5206848>
12. Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., Houlsby, N.: An image is worth 16x16 words: Transformers for image recognition at scale. *International Conference on Learning Representations* (2021), <https://openreview.net/pdf?id=YicbFdNTTy>
13. Faúndez-Zanuy, M., Mekyska, J.: Analysis of gender differences in online handwriting signals for enhancing e-health and e-security applications. *Cognitive Computation* **15**, 208–219 (2023). <https://doi.org/10.1007/s12559-023-10116-9>
14. Gattal, A., Djeddi, C., Siddiqi, I., Chibani, Y.: Gender classification from offline multi-script handwriting images using oriented basic image features (obifs). *Expert Systems with Applications* **99**, 155–167 (2018). <https://doi.org/10.1016/j.eswa.2018.01.038>
15. Hassaïne, A., Al Maadeed, S., Aljaam, J., Jaoua, A.: ICDAR 2013 competition on gender prediction from handwriting. In: *Proc. 12th Int. Conf. on Document Analysis and Recognition (ICDAR)*. pp. 1417–1421 (2013). <https://doi.org/10.1109/ICDAR.2013.286>
16. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR). pp. 770–778 (2016). <https://doi.org/10.1109/CVPR.2016.90>
17. Hinton, G., Vinyals, O., Dean, J.: Distilling the knowledge in a neural network (2015), <https://arxiv.org/abs/1503.02531>
18. Howard, A., Sandler, M., Chen, B., Wang, W., Chen, L.C., Tan, M., Chu, G., Vasudevan, V., Zhu, Y., Pang, R., Adam, H., Le, Q.: Searching for mobilenetv3. In: 2019 IEEE/CVF International Conference on Computer Vision (ICCV). pp. 1314–1324 (2019). <https://doi.org/10.1109/ICCV.2019.00140>
19. Huang, G., Liu, Z., Van Der Maaten, L., Weinberger, K.Q.: Densely Connected Convolutional Networks . In: 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR). pp. 2261–2269 (2017). <https://doi.org/10.1109/CVPR.2017.243>
20. Ibrahim, A.S., Youssef, A.E., Abbott, A.L.: Global vs. local features for gender identification using arabic and english handwriting. In: 2014 IEEE Interna-

- tional Symposium on Signal Processing and Information Technology (ISSPIT). pp. 000155–000160 (2014). <https://doi.org/10.1109/ISSPIT.2014.7300580>
21. Koepf, M., Kleber, F., Sablatnig, R.: Writer identification and writer retrieval using vision transformer for forensic documents. In: Uchida, S., Barney, E., Eglín, V. (eds.) Document Analysis Systems. pp. 352–366. Springer International Publishing, Cham (2022)
 22. Krizhevsky, A., Sutskever, I., Hinton, G.E.: Imagenet classification with deep convolutional neural networks. *Commun. ACM* **60**(6), 84–90 (2017). <https://doi.org/10.1145/3065386>
 23. Li, H., Wang, P., Shen, C., Zhang, G.: Show, attend and read: A simple and strong baseline for irregular text recognition. In: Proc. AAAI Conference on Artificial Intelligence (AAAI). AAAI’19, vol. 33, pp. 8610–8617. AAAI Press (2019). <https://doi.org/10.1609/aaai.v33i01.33018610>
 24. Li, M., Lv, T., Chen, J., Cui, L., Lu, Y., Florencio, D., Zhang, C., Li, Z., Wei, F.: Trocr: Transformer-based optical character recognition with pre-trained models. *Proceedings of the AAAI Conference on Artificial Intelligence* **37**(11), 13094–13102 (2023). <https://doi.org/10.1609/aaai.v37i11.26538>
 25. Liao, M., Zou, Z., Wan, Z., Yao, C., Bai, X.: Real-time scene text detection with differentiable binarization and adaptive scale fusion. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **45**(1), 919–931 (2023). <https://doi.org/10.1109/TPAMI.2022.3155612>
 26. Likforman-Sulem, L., Esposito, A., Faundez-Zanuy, M., Clemençon, S., Cordasco, G.: Emothaw: a novel database for emotional state recognition from handwriting and drawing. *IEEE Transactions on Human-Machine Systems* **47**(2), 273–284 (2017). <https://doi.org/10.1109/thms.2016.2635441>
 27. Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., Guo, B.: Swin Transformer: Hierarchical Vision Transformer using Shifted Windows. In: 2021 IEEE/CVF International Conference on Computer Vision (ICCV). pp. 9992–10002. IEEE Computer Society (2021). <https://doi.org/10.1109/ICCV48922.2021.00986>
 28. Liu, Z., Mao, H., Wu, C.Y., Feichtenhofer, C., Darrell, T., Xie, S.: A convnet for the 2020s. In: 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 11966–11976 (2022). <https://doi.org/10.1109/CVPR52688.2022.01167>
 29. Maadeed, S.A., Ayouby, W., Hassaïne, A., Aljaam, J.M.: Quwi: An arabic and english handwriting dataset for offline writer identification. In: 2012 International Conference on Frontiers in Handwriting Recognition. pp. 746–751 (2012). <https://doi.org/10.1109/ICFHR.2012.256>
 30. Mahmoud, S.A., Ahmad, I., Alshayeb, M., Al-Khatib, W.G., Parvez, M.T., Fink, G.A., Märgner, V., Abed, H.E.: Khatt: Arabic offline handwritten text database. In: 2012 International Conference on Frontiers in Handwriting Recognition. pp. 449–454 (2012). <https://doi.org/10.1109/ICFHR.2012.224>
 31. Marti, U.V., Bunke, H.: A full English sentence database for off-line handwriting recognition. In: Proc. 5th Int. Conf. on Document Analysis and Recognition (ICDAR). pp. 705–708 (1999). <https://doi.org/10.1109/ICDAR.1999.791885>
 32. Maurer, M.N.: Correlates of early handwriting: Differential patterns for girls and boys. *Early Education and Development* **35**, 843 – 858 (2023). <https://doi.org/10.1080/10409289.2023.2244349>
 33. Meena, R., Krishan, K., Ghosh, A., Kanchan, T.: Is it possible to estimate sex from signatures and handwriting? a review of literature, observations, and future

- perspectives. *The Science of Nature* **110**, 1–8 (2023). <https://doi.org/10.1007/s00114-023-01862-9>
34. Rabaev, I., Alkoran, I., Wattad, O., Litvak, M.: Automatic gender and age classification from offline handwriting with bilinear resnet. *Sensors* **22**(24) (2022). <https://doi.org/10.3390/s22249650>
 35. Rabaev, I., Kurar Barakat, B., Churkin, A., El-Sana, J.: The hhd dataset. In: 2020 17th International Conference on Frontiers in Handwriting Recognition (ICFHR). pp. 228–233 (2020). <https://doi.org/10.1109/ICFHR2020.2020.00050>
 36. Rabaev, I., Litvak, M., Asulin, S., Tabibi, O.H.: Automatic gender classification from handwritten images: A case study. In: Tsapatsoulis, N., Panayides, A., Theodoridis, T., Lanitis, A., Pattichis, C., Vento, M. (eds.) *Computer Analysis of Images and Patterns*. pp. 329–339. Springer International Publishing, Cham (2021)
 37. Radford, A., Kim, J.W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., Sutskever, I.: Learning transferable visual models from natural language supervision. In: Meila, M., Zhang, T. (eds.) *Proceedings of the 38th International Conference on Machine Learning. Proceedings of Machine Learning Research*, vol. 139, pp. 8748–8763. PMLR (2021), <https://proceedings.mlr.press/v139/radford21a.html>
 38. Radosavovic, I., Kosaraju, R.P., Girshick, R., He, K., Dollár, P.: Designing network design spaces. In: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 10425–10433 (2020). <https://doi.org/10.1109/CVPR42600.2020.01044>
 39. Sharma, P., Sharma, S., Jasuja, O.: Gender estimation using dynamic features of digitally captured handwriting: A pilot study. *Nowa Kodyfikacja Prawa Karnego* (2025). <https://doi.org/10.19195/2084-5065.74.2>
 40. Shi, B., Bai, X., Yao, C.: An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **39**(11), 2298–2304 (2017). <https://doi.org/10.1109/TPAMI.2016.2646371>
 41. Simonyan, K., Zisserman, A.: Very deep convolutional networks for large-scale image recognition (2015), <https://arxiv.org/abs/1409.1556>
 42. Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V., Rabinovich, A.: Going deeper with convolutions. In: 2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR). pp. 1–9 (2015). <https://doi.org/10.1109/CVPR.2015.7298594>
 43. Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., Wojna, Z.: Rethinking the inception architecture for computer vision. In: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR). pp. 2818–2826 (2016). <https://doi.org/10.1109/CVPR.2016.308>
 44. Tan, M., Le, Q.V.: Efficientnetv2: Smaller models and faster training (2021), <https://arxiv.org/abs/2104.00298>
 45. Touvron, H., Cord, M., Douze, M., Massa, F., Sablayrolles, A., Jegou, H.: Training data-efficient image Transformers & distillation through attention. In: *Proc. 38th International Conference on Machine Learning (ICML)*. PMLR, vol. 139, pp. 10347–10357. PMLR (2021). <https://doi.org/10.48550/arXiv.2012.12877>
 46. Vignesh, O., Swathy, R., Shiva, S.: Handwritten analysis for gender identification using cnn. In: 2023 4th International Conference for Emerging Technology (IN-CET). pp. 1–6 (2023). <https://doi.org/10.1109/INCET57972.2023.10170411>
 47. Xue, G., Liu, S., Gong, D., Ma, Y.: ATP-DenseNet: a hybrid deep learning-based gender identification of handwriting. *Neural Computing and Applications* **33**(10), 4611–4622 (2020). <https://doi.org/10.1007/s00521-020-05237-3>